

ENTERPRISE SECURITY BASELINE

Study Guide — RBAC, ABAC, Data Markings, Object-Level Security & Auditability

Week 1 · Day 2 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	Concepts and architecture lecture — no live Palantir platform work today
Session Format	One combined live session for India & Europe — 2:00–3:00 PM IST / 10:30–11:30 AM CEST
This Week's Lab (Day 3+)	Build a 7-object Insurance Ontology — Policyholder, Insurance Policy, Claim, Claim Document, Payment, Adjuster, Investigation — with RBAC/ABAC applied

Day 1 Recap & Coverage Check

- Covered Day 1: Data Products, Ontology-first design, Domain-Driven modeling, Object/Link Type anatomy, branching, incremental processing
- Bridged today, before Security: Functions and Action Types — introduced briefly Day 1, deepened below because today's security topics assume them

Learning Objectives

By the end of today, you will be able to:

- Distinguish RBAC (discretionary, Role-based) from ABAC (mandatory, eligibility-based)
- Apply object-, property-, and purpose-based security to an Ontology
- Explain Function and Action permissions, and how they relate to Action Types from Day 1
- Explain how security propagates across pipelines and the Ontology
- Set explicit permission boundaries for a Copilot or Agent
- Explain what auditability requires and why it's the layer that verifies all the others

1. Bridge: Functions & Action Types

Module 3's own topic list — "Function and action permissions" — assumes you already know what a Function and an Action Type are. Day 1 introduced Action Types briefly as part of Object Type anatomy; here they're deepened before security is applied to them.

Concept	Definition
Function	Business logic that takes Ontology data in and returns a computed value — an eligible claim range, a risk score — without writing anything back. Reusable across a Copilot, an Automate workflow, or an OSDK service call.

Concept	Definition
Action Type	The schema for a governed change: parameters, submission criteria, and side effects. Example — ApproveClaim: parameters (claimId, decision, amount), criteria (amount ≤ policy limit), side effect (updates Claim.status).

The security connection: who can invoke a Function, and who can trigger an Action Type, are both permission decisions — covered in Sections 5 and 6 below.

2. RBAC — Role-Based Access Control

RBAC grants permissions based on a user's assigned Role within a Project — the primary security boundary in the platform. It answers: what can this person do here?

Role	Applied to This Week's Ontology
Owner	Full control — manage the Project, its content, and who has access to it
Editor	Create and modify content — e.g. an Adjuster updating an Investigation record
Viewer	View content, no edit rights — e.g. an auditor reviewing Claim history
Discoverer	See that the Project exists and request access — cannot view content

3. ABAC — Attribute-Based Access Control, via Data Markings

A user must satisfy every attribute-based eligibility rule (a Marking) applied to a resource to access it at all — an Owner Role does not override a missing attribute match. A Marking is binary: there is no partial access.

Layer	Best For
RBAC (Roles)	What a person can generally do within a Project — coarse-grained, project-scoped
ABAC (Markings)	Categories of data needing the same protection everywhere they appear — PII, high-value claims — fine-grained, resource-scoped

Worked example: high-value claims (above a threshold amount) require a "High-Value Claim" Marking to view, regardless of Role. An Adjuster's Role grants Editor access to Claim and Investigation objects generally — but without the High-Value Marking, that same Adjuster still cannot open a high-value Claim. Both layers must pass.

4. Object & Property-Level Security

Layer	What It Controls
Object-Level Security	Whether a specific Claim, Investigation, or Payment object is visible to a given user at all — dynamic security can compute this at query time, e.g. a Claim visible only to its assigned Adjuster
Property-Level Security	Whether a specific property is visible even when the object itself is — a user might see a Claim exists without seeing its fraud-risk score

Security markings display rather than hide silently: users can see that a property is protected without seeing its value, so restrictions are transparent. Payment amounts and Claim Document contents are typical candidates for property-level protection.

5. Function & Action Permissions

Layer	Detail
Function Permissions	A Function is not automatically open — even read-only computation can expose data. Scoped by the same Role and Marking layers as object viewing.
Action Permissions	An explicit list of Roles or Markings eligible to submit a given Action Type. Submission criteria (from Section 1) combine with permission — both must pass.

Layer	Detail
Example: ApproveClaim	Only Adjuster or Senior Adjuster Roles, and only for claims within the assigned Adjuster's own caseload

6. Purpose-Based Access Patterns

Identity alone isn't always enough. The same Adjuster may be permitted to view a Claim Document for an active investigation, but not for an unrelated audit request. Purpose-based access evaluates the declared reason for access — investigation, audit, servicing — alongside Role and Marking. This matters directly for AI: an Agent's declared purpose (e.g. "fraud investigation") should scope what it's allowed to retrieve, not just its Role.

7. Security Propagation

- Across pipelines: a Marking applied to source claim data should propagate through every transform that touches it. The common failure mode is a pipeline join silently dropping a Marking, leaving a derived dataset less protected than its source.
- Across the Ontology: Link Types don't bypass security — navigating from Policyholder to Claim via a Link Type doesn't grant access to a Claim the user couldn't see directly. The same object should be equally protected whether reached directly, through a relationship, or via a derived dataset.

8. AI & Agent Permission Boundaries

Every AI capability built from Week 2 onward inherits today's boundaries — an ungoverned Ontology produces an ungoverned Copilot. Four boundaries to define explicitly for any Agent or Copilot:

Boundary	Worked Example — Week 2 Claims Copilot
Ontology Read Scope	Can read Claim, Policy, and Policyholder — not Investigation, which stays SIU-only
Action Trigger Scope	Can summarize and flag — cannot submit ApproveClaim; that stays a human-triggered Action
Tool Access Scope	Limited to the specific Functions and retrieval tools it's been granted, not the full Ontology surface
Escalation Boundary	A low-confidence response routes to a human, rather than the Copilot guessing

9. Auditability

Every audit log answers four questions: who performed an action, what the action was, when it happened, and where it occurred. Audit records are append-only — they cannot be altered after the fact — and cover both data access ("who saw this claim?") and Action submission ("who approved this payout?").

- "Who saw this claim?" — trace every access event against a specific Claim or Claim Document object
- "Who approved this payout?" — trace the ApproveClaim Action submission, including actor and state before/after
- "Did the Agent stay in scope?" — trace an Agent's read and Action-trigger events against its declared permission boundary

10. Defense in Depth — Putting It Together

No single layer covered today is sufficient on its own. A strict Role does not compensate for a missing Marking. A Marking does not compensate for weak object-level security. Enterprise deployments layer Roles, Markings, and object/property security together, then rely on audit to verify the layering is actually working as intended.

11. Glossary

Term	Meaning
RBAC	Role-Based Access Control — discretionary, granted per Project
ABAC	Attribute-Based Access Control — mandatory, eligibility-based, independent of Role

Term	Meaning
Marking	The platform's concrete mechanism implementing ABAC — a binary eligibility rule on a resource
Object-Level Security	Controls whether a user can see a given object at all
Property-Level Security	Controls whether a user can see a specific property on an object they can otherwise see
Purpose-Based Access	Access scoped by the declared reason for the request, not identity alone
Dynamic Security	Security computed at query time from the data itself
Audit Log	An append-only record of who did what, when, and where

12. Further Practice

Continue into the Day 3 self-paced lab, where you will build the full 7-object Insurance Ontology (Policyholder, Insurance Policy, Claim, Claim Document, Payment, Adjuster, Investigation), configure an incremental claims-transformation pipeline, apply RBAC and ABAC controls from today, and test restricted access to high-value and sensitive claims.